

Serie 5

Aufgabe 1:

Schreiben Sie eine Funktion `primes1(n)`, die alle Primzahlen, die kleiner-gleich $n \in \mathbb{N}$ sind, als Liste zurückgibt. Verwenden Sie dafür keine Module.

Bonus: Schreiben Sie eine Funktion `primes2(n)`, die die ersten $n \in \mathbb{N}$ vielen Primzahlen als Liste zurückgibt.

Aufgabe 2:

Schreiben Sie ein Programm, das eine natürliche Zahl $n \in \mathbb{N}, n \geq 1$, einliest und die ersten n vielen Fibonacci-Zahlen $a_0, a_1, \dots, a_{n-1}$ ausgibt. Diese sind definiert durch

- $a_0 = 1, a_1 = 1,$
- $a_k = a_{k-1} + a_{k-2}$ für $k \geq 2$.

Bsp.: Für $n = 7$ soll `1,1,2,3,5,8,13` ausgegeben werden.

Tipp: Verwenden Sie eine `for`-Schleife mit zwei Hilfsvariablen.

Aufgabe 3:

Caesar-Verschlüsselung: Schreiben Sie eine Funktion `encrypt(s)`, die einen String `s` aus Buchstaben `A, ..., Z` mit dem Caesar13-Schlüssel verschlüsselt. Das heißt, jeder Buchstabe wird um **13** Stellen im Alphabet (nach rechts) verschoben, sodass `A → N`, `B → O`, ..., `M → Z`, `N → A`, ...

Überlegen Sie sich auch, wie Sie diese Verschlüsselung umkehren können, und schreiben Sie eine entsprechende Funktion `decrypt(s)`.

Bonus: `ZVPUNRYSRVFPUYUNFTERNGUNVE`

Aufgabe 4:

Schreiben Sie eine Funktion `mc_mean`, die eine natürliche Zahl $n \in \mathbb{N}$ sowie eine optionale Anzahl $N \in \mathbb{N}, N \geq 1$, (mit Default-Wert 1000) als Eingabe erhält. Die Funktion soll nun simulieren, dass mit n 6-seitigen Würfeln gewürfelt wird und die Summe der Würfe genommen wird; dieses Würfeln soll N -mal wiederholt werden und der Durchschnitt all dieser Summen ausgegeben werden (d.h. insgesamt wird pro Funktionsaufruf nN -mal ein 6-seitiger Würfel simuliert).

Verifizieren Sie Ihr Programm folgendermaßen: Für hohe Werte von N sollte

näherungsweise der korrekte Erwartungswert $3.5n$ herauskommen.

Anmerkung: Diese Vorgehensweise heißt "Monte-Carlo-Simulation" (nach dem Glücksspielhotspot).

Aufgabe 5:

Verwenden Sie die folgende Funktion:

```
In [ ]: def spring_time(animal):  
        if animal == "cat":  
            return ["cat", "cat"]  
        elif animal == "rabbit":  
            return ["rabbit"]*10  
        elif animal == "physicist":  
            return ["physicist"]  
        else:  
            return []
```

a) Schreiben Sie eine Funktion `let_it_be_spring_list`, die eine Liste `animals_list` von Strings einliest und eine (nicht-geschachtelte) Liste ausgibt, wo auf jeden String aus `animals_list` die Funktion `spring_time` angewandt wird.

Bsp.: `animals_list = ["cat", "rabbit", "physicist"]` führt zu `['cat', 'cat', 'rabbit', 'rabbit', 'rabbit', 'rabbit', 'rabbit', 'rabbit', 'rabbit', 'rabbit', 'physicist']`.

b) Wenden Sie Ihre Funktion `let_it_be_spring_list` auf die Liste `animals_list = ["cat", "rabbit", "physicist"]` mehrfach an, d.h., auf den Output von `let_it_be_spring_time` wenden Sie wieder `let_it_be_spring_time` an; bestimmen Sie dabei in jedem Schritt die Gesamtzahl der Tiere. Finden Sie heraus, wie viele Iterationen Ihr Computer schafft, bevor Ihnen langweilig wird.

Aufgabe 6:

a) Schreiben Sie eine Funktion `let_it_be_spring_dict`, die ein Dictionary `animals_dict` von Paaren (`animal`, `count`) einliest. Die Ausgabe soll ein neues Dictionary sein, in dem die Werte `count` entsprechend der Funktion `spring_time` aktualisiert werden. Bsp.: `animals_dict = {"cat": 2, "rabbit": 2, "physicist": 2}` führt zu `animals_dict = {"cat": 4, "rabbit": 20, "physicist": 2}`. Beachten Sie, dass die Vermehrung der Anzahl aus dem Output der Funktion `spring_time` bestimmt werden soll. D.h., wenn die Funktion `spring_time` geändert wird, soll sich auch die Ausgabe von `let_it_be_spring_dict` ändern.

b) Wenden Sie Ihre Funktion `let_it_be_spring_dict` auf das Dictionary `{"cat": 1, "rabbit": 1, "physicist": 1}` mehrfach an und bestimmen Sie in jedem Schritt die Gesamtzahl der Tiere. Wieviele Iterationen schaffen Sie diesmal?